



Agendo

Tu día entre clase, trabajo y vida personal

Memoria de la Práctica 1

Mi primera app Android: de la idea al proyecto con recursos

Programación Multimedia y Dispositivos Móviles (PMDM)

Autor

Dorin Daniel Radulescu Toea

Ciclo

Desarrollo de Aplicaciones Multiplataforma · MEDAC

Repositorio

github.com/lordo0174/PMDM_P1_DanielRadulescu

Fecha

Octubre de 2026

Índice

Introducción	4
Parte A · Análisis del desarrollo móvil	5
A.1 Ventajas e inconvenientes frente a escritorio y web	5
A.2 Elección tecnológica: nativo, híbrido o multiplataforma	5
A.3 Ciclo de vida de una Activity	6
Parte B · Especificación de la app	8
B.1 Idea, público objetivo y objetivos	8
B.2 Requisitos funcionales	8
B.3 Requisitos no funcionales	8
B.4 Casos de uso	9
Parte C · Entorno de desarrollo	11
C.1 Creación del proyecto	11
C.2 Dispositivos virtuales (AVD)	11
C.3 Ejecución en los dos dispositivos	11
C.4 Estructura del proyecto	12
Parte D · Recursos de la aplicación	14
D.1 Textos	14
D.2 Internacionalización	14
D.3 Imágenes	15
D.4 Colores	15
D.5 Estilos y tema	16
D.6 Icono	17
D.7 Ampliación: tema oscuro y diseño horizontal	17
Conclusión personal	18

Índice de ilustraciones

Figura 1. Ciclo de vida de una Activity	6
Figura 2. Diagrama de casos de uso de Agendo	9
Figura 3. Agendo en el AVD Medium Phone: pantalla principal y «Nueva tarea»	11
Figura 4. Agendo en el AVD Medium Tablet en vertical	12
Figura 5. Agendo en la tablet en horizontal: diseño a dos columnas con layout-land	12
Figura 6. La app con el idioma del sistema en español y en inglés	14
Figura 7. Imágenes de res/drawable: cabecera en mapa de bits (JPG) y logotipo vectorial	15
Figura 8. Capas del icono adaptativo, resultado con dos máscaras e icono en el lanzador del emulador	17
Figura 9. Ampliación: tema oscuro (values-night) y móvil en horizontal (layout-land)	18

Índice de tablas

Tabla 1. Limitaciones del desarrollo móvil y cómo afectan a Agendo	5
Tabla 2. Comparativa de desarrollo nativo, híbrido y multiplataforma	5
Tabla 3. Requisitos funcionales	8
Tabla 4. Requisitos no funcionales	8
Tabla 5. Casos de uso de Agendo	9
Tabla 6. Ficha del caso de uso CU-02 · Crear una tarea	10
Tabla 7. Ficha del caso de uso CU-03 · Marcar una tarea como completada	10
Tabla 8. Configuración del proyecto en Android Studio	11
Tabla 9. Dispositivos virtuales (AVD) utilizados	11
Tabla 10. Imágenes de res/drawable	15
Tabla 11. Paleta de colores en el tema claro y en el oscuro	15
Tabla 12. Estilos propios y dónde se reutilizan	16

Introducción

Agendo es una app Android para planificar las tareas del día, pensada para estudiantes de FP que compaginan las clases con un trabajo y su vida personal. Muestra cuántas tareas quedan pendientes, su prioridad por colores y permite filtrarlas por categoría (Clase, Trabajo y Personal).

La app está hecha en Java con vistas XML, a partir de la plantilla Empty Views Activity y con API mínima 24. La práctica no pedía programar funcionalidad, así que el trabajo se centra en dos pantallas (la lista de tareas de hoy y el formulario "Nueva tarea") y en organizar todos sus recursos en `res/`.

La memoria sigue el orden del enunciado: análisis del desarrollo móvil (Parte A), especificación de la app (Parte B), entorno de desarrollo (Parte C) y recursos de la aplicación (Parte D). El proyecto completo está en el repositorio [PMDM_P1_DanielRadulescu](#).

Parte A · Análisis del desarrollo móvil

A.1 Ventajas e inconvenientes frente a escritorio y web

Un planificador tiene sentido sobre todo en el móvil, porque es el dispositivo que llevo siempre encima. A cambio, el móvil tiene recursos más limitados y una variedad de dispositivos mucho mayor que un ordenador.

Ventajas del desarrollo móvil

- **Siempre a mano:** puedo apuntar una tarea en el momento, entre clases o en el autobús, sin encender un ordenador.
- **Acceso al hardware y al sistema:** notificaciones, alarmas, calendario, cámara o sensores, que en una web están muy limitados.
- **Avisos aunque la app esté cerrada:** el sistema puede mostrar un recordatorio a la hora indicada.
- **Funciona sin conexión:** los datos pueden guardarse en el propio dispositivo.
- **Distribución sencilla:** Google Play instala y actualiza la app automáticamente.

Inconvenientes

- Hay que diseñar para pantallas pequeñas y táctiles, y adaptarse a tablets y a la rotación.
- El sistema controla el ciclo de vida: puede pausar o cerrar la app en cualquier momento.
- Hay que probar en muchas versiones de Android, fabricantes y densidades de pantalla.
- Publicar requiere pasar la revisión de la tienda; en la web basta con subir los cambios al servidor.

Tabla 1. Limitaciones del desarrollo móvil y cómo afectan a Agendo

Limitación	Qué supone	Cómo afecta a Agendo
Batería	Los procesos en segundo plano la gastan y Android los restringe (modo Doze).	Los recordatorios se programarían con alarmas del sistema en vez de un servicio siempre activo. El tema oscuro ahorra energía en pantallas OLED.
Memoria RAM	Si falta memoria, el sistema puede cerrar la app cuando está en segundo plano.	La imagen de cabecera es pequeña (720 × 320 px, 35 KB), los iconos son vectoriales y el estado de las tareas se recupera al recrear la pantalla.
Conectividad	La conexión puede fallar o no existir (metro, zonas sin cobertura).	Agendo no necesita Internet: todo funciona en local. Una futura sincronización en la nube debería esperar a tener red.
Tamaño de pantalla	Poco espacio y control con el dedo.	Una sola acción principal (botón flotante "Nueva tarea"), zonas táctiles de al menos 48 dp, textos en <code>sp</code> y un diseño a dos columnas en horizontal y en tablet.
Fragmentación	Muchas versiones de Android, fabricantes y densidades.	API mínima 24, iconos vectoriales que valen para todas las densidades, icono por densidad en <code>mipmap-*</code> y pruebas en dos AVD distintos.
Almacenamiento	Muchos usuarios tienen poco espacio libre.	Se priorizan vectores y JPG comprimido; el APK de depuración ocupa 8,2 MB.

A.2 Elección tecnológica: nativo, híbrido o multiplataforma

Agendo se desarrolla en **nativo Android con Java y vistas XML**: es la opción con mejor rendimiento y acceso completo al sistema, y su mayor inconveniente (tener que repetir la app para iOS) no afecta a esta práctica.

Tabla 2. Comparativa de desarrollo nativo, híbrido y multiplataforma

	Nativo	Híbrido	Multiplataforma
Lenguajes	Java o Kotlin (Android); Swift (iOS)	HTML, CSS y JavaScript dentro de un WebView	Dart (Flutter), JavaScript/TypeScript (React Native), C# (.NET MAUI)
Rendimiento	El mejor: el código se compila para la plataforma	El más bajo: la interfaz se dibuja en un navegador interno	Cercano al nativo

	Nativo	Híbrido	Multiplataforma
Acceso al hardware	Total y directo a todas las API del sistema	A través de plugins	Casi total; lo más específico necesita código nativo
Coste	Alto si se quiere Android e iOS (dos códigos distintos)	Bajo: un código web para todo	Medio: un código para las dos plataformas
Ejemplo de framework	Android SDK con Android Studio (Jetpack)	Ionic con Capacitor (o Apache Cordova)	Flutter o React Native

Por qué nativo Android:

- **Solo necesito Android:** la app va dirigida a usuarios de Android, así que el ahorro del multiplataforma no compensa.
- **Gestión de recursos de Android:** el objetivo de la práctica es trabajar con `strings.xml`, `values-en`, `values-night`, `layout-land` o `mipmap-*`, que son propios del desarrollo nativo.
- **Integración con el sistema:** los recordatorios, el icono adaptativo o el tema oscuro funcionan directamente, sin plugins intermedios.
- **Rendimiento y batería:** una app que se abre muchas veces al día tiene que arrancar rápido y gastar poco.
- **Aprendizaje:** conocer la plataforma base ayuda a entender después cómo funcionan los frameworks multiplataforma.

A.3 Ciclo de vida de una Activity

Una Activity pasa por tres estados mientras existe: **visible y activa** (en primer plano), **visible pero en pausa** y **oculta** (en segundo plano). Android avisa de cada cambio llamando a un método: `onCreate`, `onStart`, `onResume`, `onPause`, `onStop`, `onRestart` y `onDestroy`.

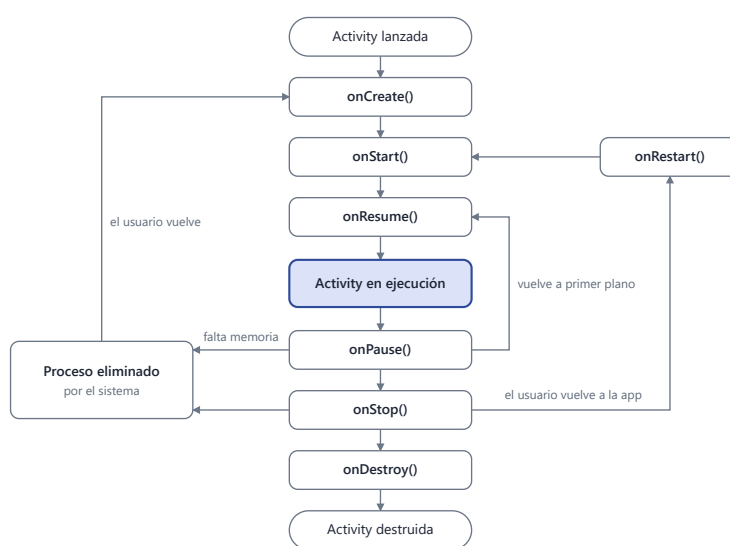


Figura 1. Ciclo de vida de una Activity

1. El usuario recibe una llamada mientras mira la lista de tareas y la contesta:

- La pantalla de llamada se pone delante. `MainActivity` pasa por `onPause()` y `onStop()`: queda en segundo plano, pero **no se destruye**.
- Al colgar y volver a Agendo se ejecutan `onRestart()`, `onStart()` y `onResume()`. La lista aparece igual que estaba: mismas casillas marcadas y mismo filtro.
- Si durante la llamada el sistema necesitara memoria podría cerrar el proceso. Al volver, la Activity se crearía de nuevo con `onCreate()` y Android restauraría el estado guardado.

2. El usuario gira el móvil:

- Es un cambio de configuración, así que Android **destruye y vuelve a crear** la Activity: `onPause()` → `onStop()` → `onSaveInstanceState()` → `onDestroy()` y después `onCreate()` → `onStart()` → `onRestoreInstanceState()` → `onResume()`.
- En el nuevo `onCreate()` Android carga `layout-land/activity_main.xml`, el diseño a dos columnas.
- Las casillas y el chip de categoría conservan su estado porque tienen `id`. En `onRestoreInstanceState()` llamo a `actualizarTareas()` para recalculer el resumen ("Tienes 3 tareas pendientes") con esos estados.

3. El usuario pulsa el botón Atrás:

- **En "Nueva tarea":** `NuevaTareaActivity` se cierra con `onPause()` → `onStop()` → `onDestroy()` y lo escrito en el formulario se descarta. La pantalla principal vuelve con `onRestart()` → `onStart()` → `onResume()`.
- **En la pantalla principal:** desde Android 12, la Activity raíz no se destruye al pulsar Atrás. Pasa a segundo plano (`onPause()` → `onStop()`) y, si el usuario vuelve a abrir Agendo, continúa con `onRestart()`. En versiones anteriores se cerraba con `onDestroy()`.

Parte B · Especificación de la app

B.1 Idea, público objetivo y objetivos

Idea. Agendo es un planificador de tareas centrado en el día de hoy. Muchas apps de tareas son genéricas o tienen tantas opciones que cuesta usarlas a diario. Agendo se limita a lo importante: las tareas del día, tres categorías fijas (Clase, Trabajo y Personal) y una prioridad que se ve por colores. La idea es que apuntar o tachar una tarea lleve unos segundos y se pueda hacer con una mano.

Público objetivo. Estudiantes de FP y universitarios de 16 a 30 años que compaginan los estudios con un trabajo a tiempo parcial. Tienen un móvil Android de gama media, poco tiempo libre y muchas fechas que recordar: entregas, exámenes, turnos y planes personales.

Objetivos medibles

1. **Crear tareas rápido:** un usuario nuevo crea una tarea con título y categoría en menos de 30 segundos y con un máximo de 4 toques desde la pantalla principal.
2. **Ver el día de un vistazo:** el resumen de tareas pendientes se ve sin desplazarse, y 9 de cada 10 usuarios de prueba dicen cuántas tareas les quedan en menos de 5 segundos.
3. **Cumplir los plazos:** tras 4 semanas de uso, el usuario marca como completadas al menos el 80 % de sus tareas antes de su fecha límite.

B.2 Requisitos funcionales

Tabla 3. Requisitos funcionales

ID	Requisito	Prioridad	Estado en el prototipo
RF-01	La app muestra las tareas del día con su título, categoría, hora y prioridad.	Alta	Funciona
RF-02	El usuario puede crear una tarea con título (obligatorio), descripción, categoría, fecha límite, prioridad y recordatorio.	Alta	Solo interfaz
RF-03	El usuario puede marcar y desmarcar una tarea como completada; al marcarla, su título se tacha.	Alta	Funciona
RF-04	La pantalla principal muestra el número de tareas pendientes y una barra de progreso con las completadas.	Media	Funciona
RF-05	El usuario puede filtrar la lista por categoría: Todas, Clase, Trabajo o Personal.	Media	Funciona
RF-06	Si el recordatorio está activado, la app avisa con una notificación una hora antes de la fecha límite.	Media	Pendiente
RF-07	Las tareas se guardan en el dispositivo y se conservan al cerrar la app.	Alta	Pendiente

RF-02 valida el título y confirma el guardado, pero no añade la tarea a la lista. RF-06 y RF-07 necesitan guardar datos y programar avisos, que quedan fuera de esta práctica.

B.3 Requisitos no funcionales

Tabla 4. Requisitos no funcionales

ID	Tipo	Requisito	Cómo se verifica
RNF-01	Usabilidad	Abrir el formulario de nueva tarea, marcar una tarea o filtrar la lista requiere un solo toque desde la pantalla principal, y todas las zonas táctiles miden al menos 48 × 48 dp.	Contando los toques de cada acción y midiendo los elementos con el Layout Inspector de Android Studio.

ID	Tipo	Requisito	Cómo se verifica
RNF-02	Compatibilidad	La app funciona en Android 7.0 (API 24) o superior. API 24 llega a casi todos los móviles en uso (alrededor del 98 %, según el asistente de Android Studio) y es compatible con AndroidX y Material 3. Subir a API 26 dejaría fuera Android 7 sin aportar nada que Agendo necesite.	<code>minSdk = 24</code> en <code>app/build.gradle.kts</code> y ejecución en un AVD con API 24.
RNF-03	Rendimiento	En un arranque en frío, la pantalla principal aparece en menos de 2 segundos en un móvil de gama media (AVD Medium Phone).	Tiempo que muestra Logcat en la línea "Displayed ..." o el comando <code>adb shell am start -W</code> .
RNF-04	Internacionalización	Ningún texto está escrito en el código ni en los layouts, y la app se muestra completa en español y en inglés según el idioma del sistema.	Lint sin avisos "Hardcoded text" ni "Missing translation", y cambiando el idioma del emulador.
RNF-05	Accesibilidad	Los textos tienen un contraste mínimo de 4,5:1 con su fondo, sus tamaños están en <code>sp</code> y todas las imágenes tienen <code>contentDescription</code> .	Accessibility Scanner y Lint.
RNF-06	Tamaño	El APK de depuración ocupa menos de 10 MB (ahora 8,2 MB).	Tamaño de <code>app-debug.apk</code> tras compilar.

B.4 Casos de uso

Agendo tiene dos actores: el **Usuario**, que hace casi todo, y el **Sistema Android**, que es quien lanzaría los recordatorios a su hora.



Figura 2. Diagrama de casos de uso de Agendo

Tabla 5. Casos de uso de Agendo

Caso de uso	Actor	Descripción
CU-01 Consultar las tareas del día	Usuario	Ver la lista de hoy y el resumen de pendientes.
CU-02 Crear una tarea	Usuario	Rellenar el formulario "Nueva tarea" y guardarla.
CU-03 Marcar una tarea como completada	Usuario	Marcar o desmarcar una tarea de la lista.
CU-04 Filtrar por categoría	Usuario	Ver solo las tareas de Clase, Trabajo o Personal.
CU-05 Recibir un recordatorio	Sistema Android	Avisar una hora antes de la fecha límite.

Tabla 6. Ficha del caso de uso CU-02 · Crear una tarea

Campo	Descripción
Actor	Usuario
Precondición	La app está abierta en la pantalla principal.
Flujo principal	1. El usuario pulsa "Nueva tarea". 2. La app muestra el formulario, con prioridad Media y el recordatorio activado por defecto. 3. El usuario escribe el título y, si quiere, la descripción, la categoría, la fecha límite y la prioridad. 4. Pulsa "Guardar tarea". 5. La app muestra el mensaje «Tarea "..." guardada» y vuelve a la lista.
Flujo alternativo	4a. El título está vacío: la app marca el campo en rojo con "Escribe un título para la tarea" y no guarda; el usuario lo escribe y vuelve al paso 4. 3b. El usuario pulsa Cancelar, la flecha o Atrás: vuelve a la lista sin guardar nada.
Postcondición	La tarea queda creada (en el prototipo solo se confirma con el mensaje) y se muestra la pantalla principal.

Tabla 7. Ficha del caso de uso CU-03 · Marcar una tarea como completada

Campo	Descripción
Actor	Usuario
Precondición	Hay al menos una tarea pendiente visible en la lista.
Flujo principal	1. El usuario toca la casilla de la tarea o la tarjeta entera. 2. La app marca la casilla y tacha el título. 3. La app actualiza el resumen: una pendiente menos, la barra avanza y cambia "Completadas: X de Y".
Flujo alternativo	1a. La tarea ya estaba completada: se desmarca, se quita el tachado y el número de pendientes sube. 1b. La tarea no se ve por el filtro activo: el usuario elige "Todas" o su categoría y continúa en el paso 1.
Postcondición	El estado de la tarea y el resumen coinciden, y se mantienen aunque se gire la pantalla.

Parte C · Entorno de desarrollo

C.1 Creación del proyecto

El proyecto se creó en Android Studio con la plantilla **Empty Views Activity** y esta configuración:

Tabla 8. Configuración del proyecto en Android Studio

Opción	Valor
Nombre	PMDM_P1_DanielRadulescu
Paquete	es.medac.danielradulescu.app
Lenguaje	Java
API mínima (minSdk)	24 (Android 7.0)
API de compilación y objetivo	37
Configuración de compilación	Kotlin DSL (build.gradle.kts), con catálogo de versiones libs.versions.toml

C.2 Dispositivos virtuales (AVD)

Creé dos AVD con densidades distintas para comprobar que la interfaz se adapta al tamaño de pantalla:

Tabla 9. Dispositivos virtuales (AVD) utilizados

AVD	Tipo	Resolución	Densidad	Tamaño en dp	API
Medium Phone	Móvil	1080 × 2400 px	420 dpi (xxhdpi)	411 × 914 dp	37
Medium Tablet	Tablet	2560 × 1600 px	320 dpi (xhdpi)	1280 × 800 dp	37

C.3 Ejecución en los dos dispositivos

En el móvil la pantalla principal es una sola columna con desplazamiento. En la tablet en vertical cabe todo sin desplazarse, y en horizontal Android usa `layout-land` y muestra dos columnas: la cabecera con el resumen a la izquierda y la lista de tareas a la derecha.

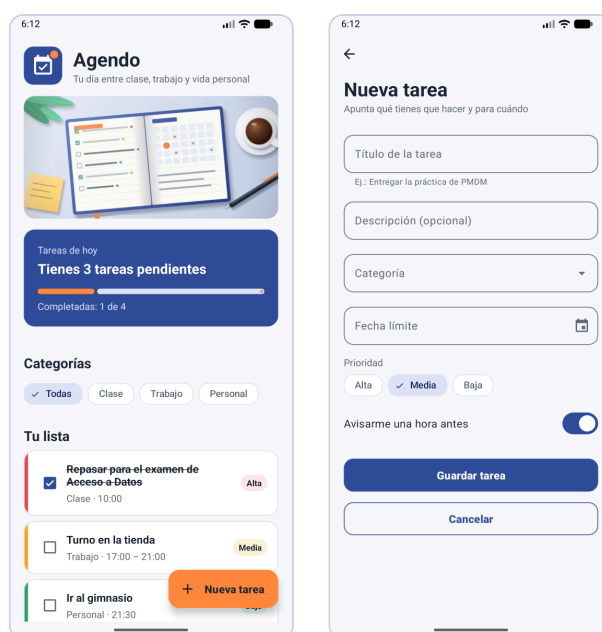


Figura 3. Agendo en el AVD Medium Phone: pantalla principal y «Nueva tarea»

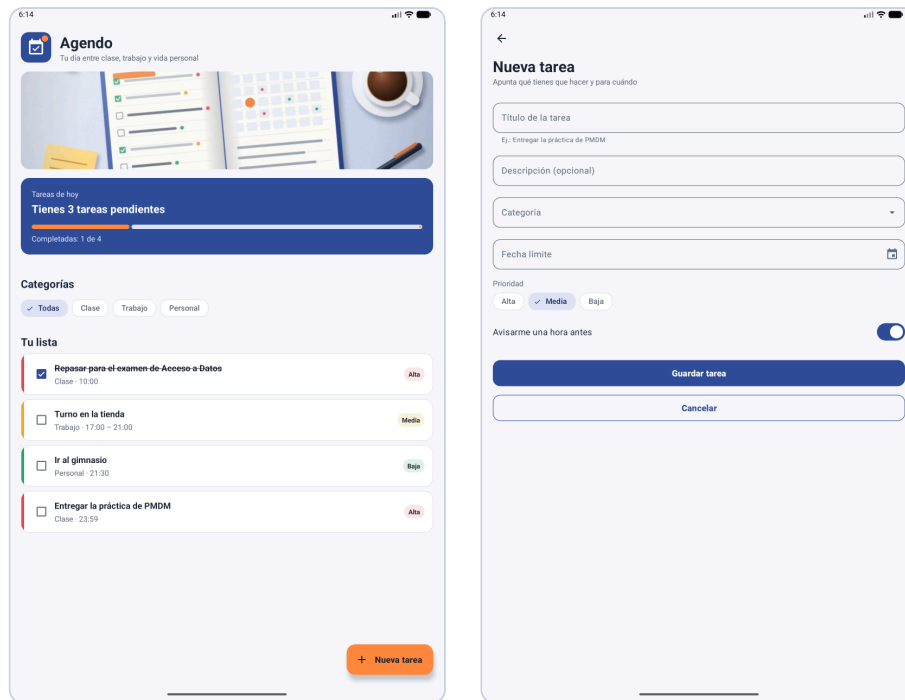


Figura 4. Agendo en el AVD Medium Tablet en vertical

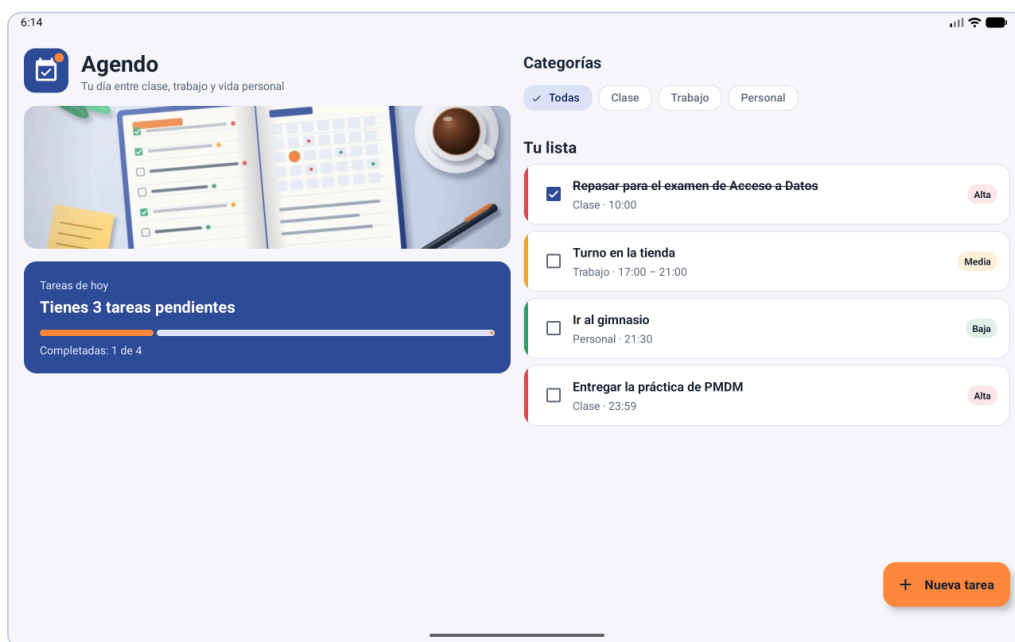


Figura 5. Agendo en la tablet en horizontal: diseño a dos columnas con layout-land

C.4 Estructura del proyecto

- **AndroidManifest.xml** : describe la app ante el sistema: nombre (`@string/app_name`), icono (`@mipmap/ic_launcher`), tema (`@style/Theme.Agendo`) y las dos Activities. `MainActivity` tiene el `intent-filter` `MAIN/LAUNCHER` que la hace aparecer en el lanzador; `NuevaTareaActivity` declara a `MainActivity` como su pantalla padre.
- **java/** : el código Java organizado por paquete: `MainActivity` y `NuevaTareaActivity` , además de las carpetas de pruebas `test/` y `androidTest/` .
- **res/** : todos los recursos, separados por tipo (`layout` , `values` , `drawable` , `mipmap` , `color`) y con calificadores que Android elige solo: `values-en` para el inglés, `values-night` para el tema oscuro y `layout-land` para el horizontal.
- **Ficheros Gradle:**

- `settings.gradle.kts` : nombre del proyecto, módulos y repositorios.
- `build.gradle.kts` de la raíz: plugins comunes.
- `app/build.gradle.kts` : `namespace` , `applicationId` , versiones de SDK y dependencias (AppCompat, Material, ConstraintLayout).
- `gradle/libs.versions.toml` : versiones de las librerías en un solo sitio.
- `gradlew` y `gradle/wrapper/` : descargan la versión correcta de Gradle para compilar.

Relación entre la Activity y su layout. El layout XML define qué vistas hay y cómo se ven; la Activity en Java decide qué hacen. Al compilar, Android genera la clase `R` , con un identificador para cada recurso. En `onCreate()` , `setContentView(R.layout.activity_main)` *infla* el XML y crea las vistas, y después `findViewById(R.id.grupoCategorias)` devuelve una de ellas para darle comportamiento, por ejemplo un listener. Android elige solo entre `layout/` y `layout-land/` según la orientación; como los `id` son los mismos en los dos, el código no cambia. En el XML, `tools:context=".MainActivity"` indica al editor de diseño a qué Activity pertenece.

Parte D · Recursos de la aplicación

Las dos pantallas usan solo recursos de `res/values`: en los layouts no hay ningún texto, color ni medida escritos a mano, y Lint no muestra avisos de "Hardcoded text".

D.1 Textos

`values/strings.xml` tiene **40 textos**, además de un `string-array` y un `plurals`:

- **string-array categorías**: sus elementos apuntan a otros textos (`@string/categoria_clase ...`) y rellena el desplegable "Categoría" directamente desde el XML con `app:simpleItems="@array/categorias"`.
- **Textos con parámetro**: `mensaje_tarea_guardada` (Tarea «%1\$s» guardada) recibe el título escrito, y `resumen_progreso` (Completadas: %1\$d de %2\$d) recibe dos números. En Java se rellenan con `getString(R.string.mensaje_tarea_guardada, titulo)`.
- **plurals resumen_pendientes**: muestra "1 tarea pendiente" o "3 tareas pendientes" según el número, con `getQuantityString()`.
- **app_name**: lleva `translatable="false"` porque "Agendo" es una marca y no se traduce.

```
<string name="mensaje_tarea_guardada">Tarea «%1$s» guardada</string>
<string-array name="categorias">
  <item>@string/categoria_clase</item>
  <item>@string/categoria_trabajo</item>
  <item>@string/categoria_personal</item>
</string-array>
```

D.2 Internacionalización

La traducción al inglés está en `values-en/strings.xml`, con los mismos nombres de texto. Si el idioma del sistema es inglés, Android usa esa carpeta; si es cualquier otro, usa `values/`, que está en español. Las horas se adaptan al formato inglés de 12 horas ("5:00 – 9:00 PM") y la fecha a `mm/dd/yyyy`. El `string-array` no hace falta copiarlo: como apunta a textos que sí están traducidos, el desplegable sale en inglés automáticamente.

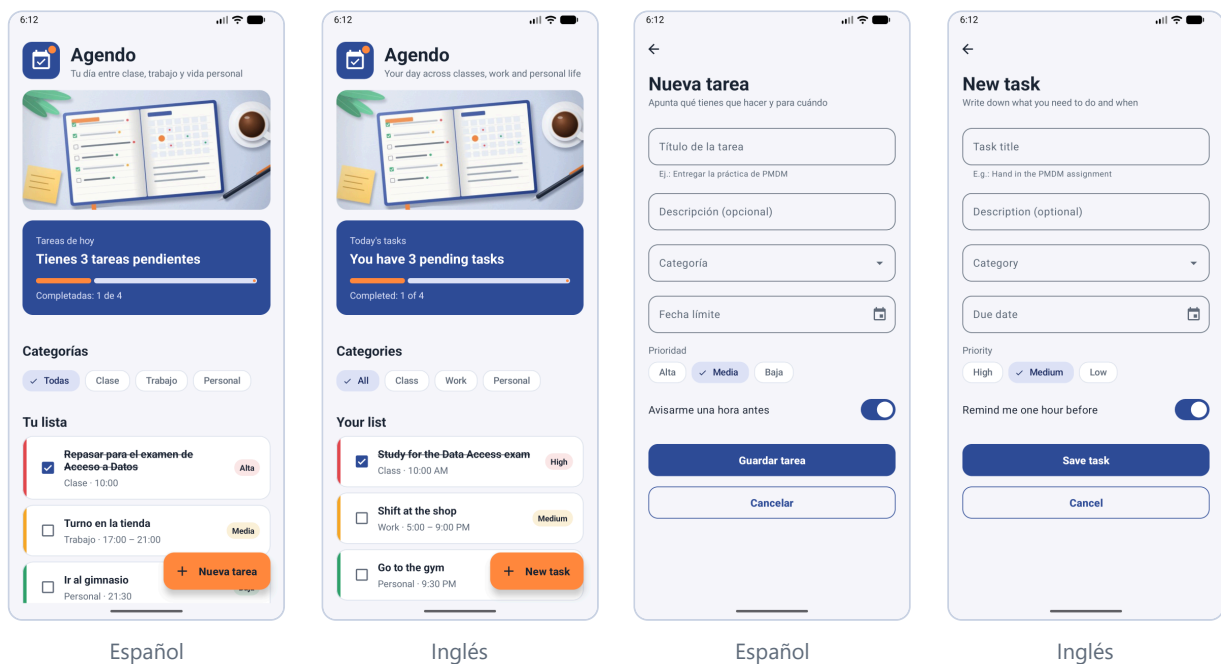
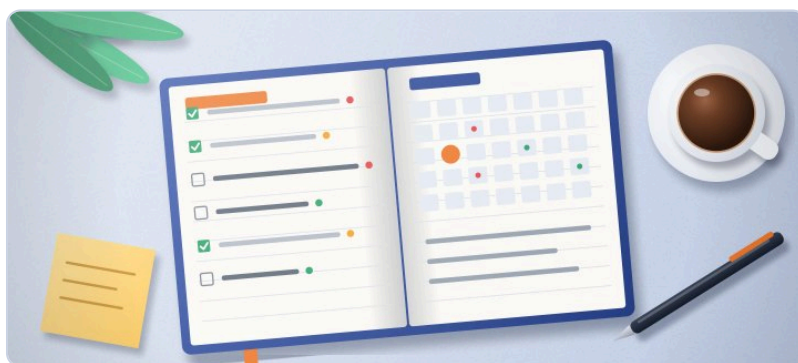


Figura 6. La app con el idioma del sistema en español y en inglés

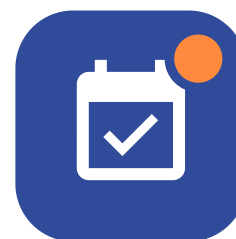
D.3 Imágenes

Tabla 10. Imágenes de res/drawable

Imagen	Formato	Uso
ic_add, ic_calendario, ic_volver	Vectorial (Vector Asset, Material Icons)	Botón "Nueva tarea", campo de fecha y flecha para volver
ic_logo_agendo	Vectorial (propio)	Logotipo de la cabecera
cabecera_escritorio.jpg	Mapa de bits (JPG, 720 × 320 px, 35 KB)	Ilustración de un escritorio con una agenda abierta



cabecera_escritorio.jpg · 720 × 320 px · 35 KB



ic_logo_agendo.xml · vectorial

Figura 7. Imágenes de res/drawable: cabecera en mapa de bits (JPG) y logotipo vectorial

Cuándo conviene cada formato. Un vector guarda instrucciones de dibujo, así que ocupa menos de 1 KB, se ve nítido a cualquier tamaño y **un solo archivo sirve para todas las densidades**. Es ideal para iconos y formas planas. Un mapa de bits guarda píxeles: se pixela al ampliarlo y, para verse bien en todas las pantallas, necesitaría una copia por densidad, lo que aumenta el tamaño de la app. Compensa en fotos o imágenes con degradados, sombras y texturas, como la cabecera, que como vector necesitaría miles de trazados y pesaría más que el JPG.

Lint avisa (`IconLocation`) de que los mapas de bits deberían ir en carpetas por densidad. La práctica pedía `res/drawable` , así que la imagen se hizo pequeña para que el escalado automático no gaste memoria de más.

D.4 Colores

La paleta está en `colors.xml` con nombres de función, y `values-night/colors.xml` redefine los mismos nombres para el tema oscuro.

Tabla 11. Paleta de colores en el tema claro y en el oscuro

Nombre	Claro	Oscuro	Función
color_primario	 #2F4B9A	 #A9BCFF	Color de marca: botón Guardar, casillas, tarjeta de resumen, logotipo
color_sobre_primario	 #FFFFFF	 #0F1B4D	Texto sobre el color primario
color_primario_contenedor	 #DCE3F9	 #263A7A	Chip seleccionado y fondo de la barra de progreso
color_acento	 #FF8A3D	 #FF9A57	Botón "Nueva tarea" y progreso
color_fondo	 #F5F7FB	 #11141B	Fondo de las pantallas
color_superficie	 #FFFFFF	 #1A1E27	Tarjetas
color_borde	 #DDE3EE	 #2E3442	Borde de las tarjetas
color_texto_principal	 #1C2333	 #E6E9F0	Textos principales
color_texto_secundario	 #5F6B7A	 #A3ACBB	Detalles, ayudas y subtítulos

Nombre	Claro	Oscuro	Función
color_prioridad_alta	 #E5484D	 #FF6B6F	Franja de prioridad alta
color_prioridad_media	 #F5A524	 #FFB547	Franja de prioridad media
color_prioridad_baja	 #30A46C	 #4CC38A	Franja de prioridad baja

Hay 6 colores más de apoyo: los textos sobre el contenedor y el acento, una superficie secundaria y los fondos suaves de las tres etiquetas de prioridad.

Justificación de la paleta.

- **Azul índigo:** transmite calma, orden y concentración, que es lo que busca alguien que se organiza.
- **Naranja:** añade energía y se reserva para la acción principal, para que destaque.
- **Rojo, ámbar y verde:** las prioridades usan el código de un semáforo, que se entiende sin leer.
- **Fondo y textos:** un gris azulado muy claro con tarjetas blancas separa las tareas sin recargar, y los textos superan el contraste mínimo de 4,5:1.

D.5 Estilos y tema

`styles.xml` define estilos propios que se reutilizan en las dos pantallas. Usan dos formas de herencia: **por el nombre** (`Agendo.Texto.Secundario` hereda de `Agendo.Texto` por el punto) y **con parent** (`Agendo.Boton.Principal` hereda de `Widget.Material3.Button`).

Tabla 12. Estilos propios y dónde se reutilizan

Estilo	Hereda de	Dónde se usa
<code>Agendo.Texto.Secundario</code>	<code>Agendo.Texto</code>	Eslogan, detalles de las tareas, subtítulos (7 veces)
<code>Agendo.Chip</code>	<code>Widget.Material3.Chip.Filter</code>	4 chips de categoría y 3 de prioridad (7 veces)
<code>Agendo.Tarjeta</code>	<code>Widget.Material3.CardView.Outlined</code>	Las 4 tarjetas de tarea; <code>Agendo.Tarjeta.Resumen</code> hereda de él
<code>Agendo.Texto.TituloTarea</code>	<code>Agendo.Texto</code>	Título de cada tarea (4 veces)
<code>Agendo.Texto.Etiqueta.*</code> y <code>Agendo.MarcaPrioridad.*</code>	<code>Agendo.Texto</code> y estilo base propio	Etiqueta y franja de color de cada prioridad
<code>Agendo.CampoTexto</code>	<code>Widget.Material3.TextInputLayout.OutlinedBox</code>	Campos del formulario (3 veces, más la variante <code>.Desplegable</code>)
<code>Agendo.Texto.TituloPantalla</code>	<code>Agendo.Texto</code>	Título de las dos pantallas
<code>Agendo.Boton.Principal</code> y <code>.Secundario</code>	Botones de Material 3	Guardar tarea y Cancelar
<code>Agendo.BotonFlotante</code>	<code>Widget.Material3.ExtendedFloatingActionButton</code>	Botón "Nueva tarea"

Tema. `Theme.Agendo` hereda de `Theme.Material3.DayNight.NoActionBar` y sustituye sus colores por los de la paleta (`colorPrimary`, `colorSecondary`, `android:colorBackground`, `colorSurface`...). Así, los componentes que no tienen estilo propio, como la casilla o el interruptor, también usan los colores de `Agendo`. Se aplica a toda la app en el `Manifest` con `android:theme`.

Medidas. Todas están en `dimens.xml`: márgenes y tamaños en `dp` (`margen_pantalla` 20 dp, `radio_tarjeta` 16 dp, `alto_boton` 56 dp) y textos en `sp` (de 12 a 28 sp). Los `dp` hacen que un elemento mida lo mismo en cualquier densidad, y los `sp` además respetan el tamaño de letra que el usuario elige en Ajustes.

D.6 Icono

El icono es **adaptativo**: un calendario con un check y un punto naranja de aviso sobre el índigo de la marca, igual que el logotipo. Tiene tres capas:

- **Primer plano:** `drawable/ic_launcher_foreground.xml`, vectorial, dibujado dentro de la zona segura de 66 dp para que no se corte con ninguna máscara.
- **Fondo:** el color `ic_launcher_background`.
- **Monocroma:** `drawable/ic_launcher_monochrome.xml`, para los iconos temáticos de Android 13 o superior.

`mipmap-anydpi-v26/ic_launcher.xml` une las tres capas para Android 8 o superior. Las carpetas `mipmap-mdpi` a `mipmap-xxxhdpi` tienen las versiones PNG, de 48 a 192 px, para Android 7. En el Manifest se aplica con `android:icon="@mipmap/ic_launcher"` y `android:roundIcon="@mipmap/ic_launcher_round"`. El diseño original en SVG está en `docs/icono`.

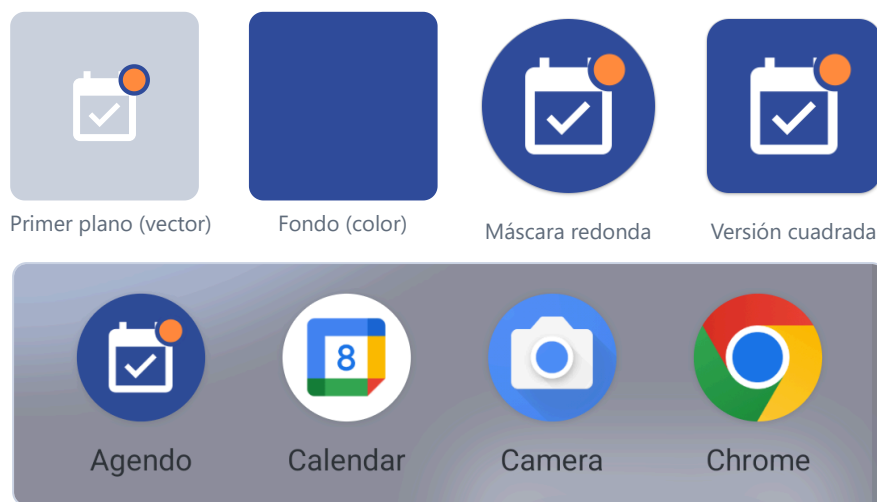


Figura 8. Capas del icono adaptativo, resultado con dos máscaras e icono en el lanzador del emulador

D.7 Ampliación: tema oscuro y diseño horizontal

- **Tema oscuro:** `values-night/colors.xml` redefine los mismos nombres de color con tonos para fondo oscuro. El tema y los estilos no cambian: Android elige este archivo cuando el modo oscuro está activado. Un selector de color (`color/borde_campo_texto.xml`) mantiene visibles los bordes de los campos en los dos temas.
- **Horizontal:** `layout-land/activity_main.xml` organiza la pantalla principal en dos columnas que se desplazan por separado. Para no repetir el diseño, las dos orientaciones reutilizan las mismas secciones con `<include>` (`seccion_cabecera.xml` y `seccion_tareas.xml`).

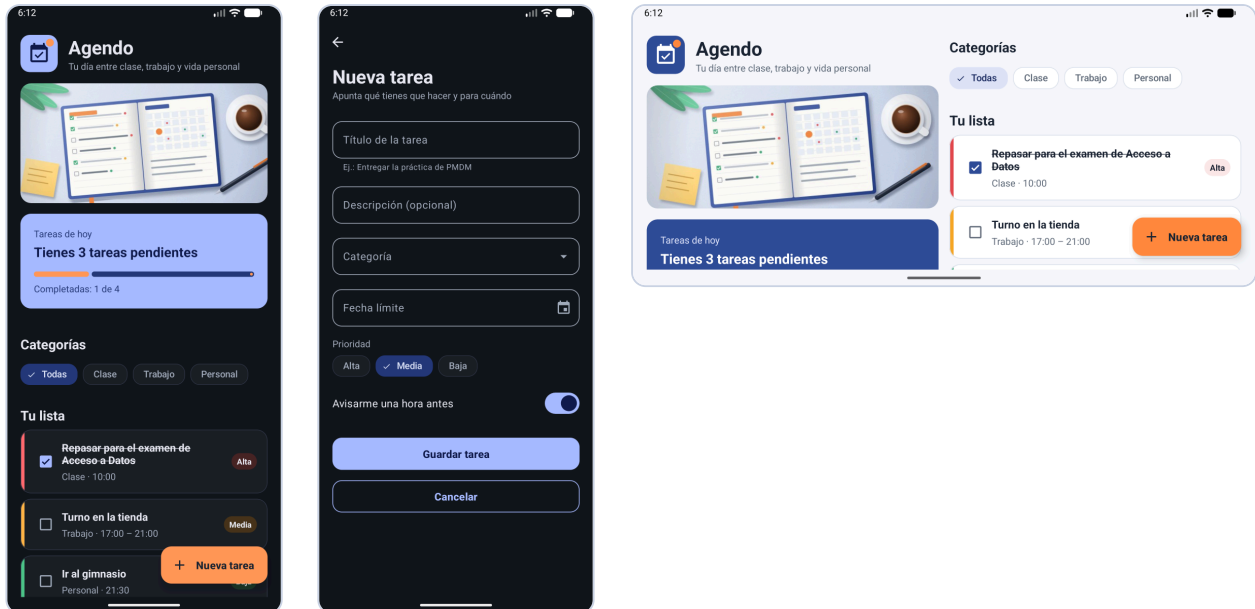


Figura 9. Ampliación: tema oscuro (values-night) y móvil en horizontal (layout-land)

Conclusión personal

Con esta práctica he aprendido que una app Android no es solo código Java: gran parte del trabajo está en `res/`. Al separar los textos, colores, medidas y estilos del diseño he podido traducir la app al inglés y añadir un tema oscuro y un diseño horizontal sin tocar el código, solo añadiendo carpetas con calificadores (`values-en`, `values-night`, `layout-land`). También entiendo mejor el ciclo de vida de una Activity, sobre todo lo que pasa al girar la pantalla, y cuándo conviene una imagen vectorial o un mapa de bits.

Dificultades que he tenido:

- **Colores:** encontrar una paleta con buen contraste que funcionara también en modo oscuro. Los bordes de los campos de texto desaparecían y tuve que usar un selector de color.
- **Avisos de Lint:** el plural en español pide una forma `many`, y el aviso `InefficientWeight` sale al usar `@dimen` en lugar de escribir `0dp`.
- **Emulador:** a veces se quedaba "offline" y tuve que arrancarlo en frío. Además tenía fijado el español como idioma de la app, y por eso al principio no veía la traducción.
- **Git:** organizar el trabajo en commits pequeños y con mensajes descriptivos.

Como mejora me gustaría guardar las tareas en una base de datos (Room) y programar los recordatorios con notificaciones, que son los requisitos RF-06 y RF-07 que han quedado pendientes.